KEEL

Market Perception Index (MPI)

Architecture, Use Cases & User Scenarios

Current backend architecture after refactor Phases 0-8

Phase 9 final independent review remains a separate acceptance step.

Purpose A clean technical reference for how KEEL computes, stores, secures and exposes MPI summaries and history - plus the user journeys the subsystem supports.

1. Document Map

Section	What it answers
2. Executive Overview	What MPI is and which product behavior is currently preserved.
3. Final Architecture	How delivery, application, orchestration, deterministic logic and PostgreSQL composition fit together.
4. Request Flows	What happens during summary and history requests, including commits and evidence persistence.
5. Decision Semantics	When MPI returns UNKNOWN versus FACT and how low-sample behavior works.
6. Use Cases	The bounded business/application operations supported by the subsystem.
7. User Scenarios	Concrete step-by-step examples for business, product, competitor, low-data, history and isolation cases.
8. Security & Transaction Invariants	What future refactors must not accidentally break.
9. Reference Pattern Guidance	What other KEEL features should reuse - and what is MPI-specific legacy behavior.

2. Executive Overview

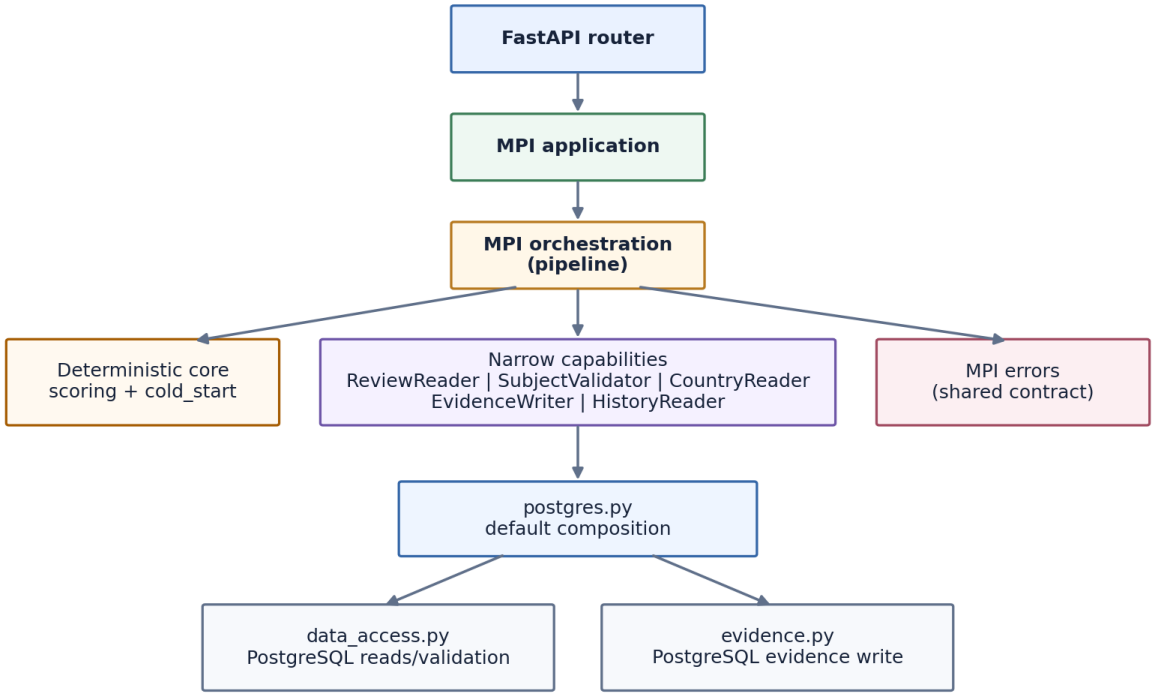
MPI is a bounded KEEL backend feature that turns eligible analyzed review evidence into a tenant-scoped market-perception result. It supports BUSINESS, PRODUCT and COMPETITOR subjects, persists an evidence record when a summary is requested, and exposes historical evidence through a read-only history path.

- Zero usable analyzed reviews produce UNKNOWN evidence and no numeric MPI.
- A small usable sample still produces FACT evidence with a numeric MPI plus a low-sample status.
- A sufficient sample produces FACT evidence with a numeric MPI.
- Repeated summary requests may append additional evidence/history rows; no deduplication was introduced.
- All characterized tenant roles continue to use the same authorization and entitlement authorities.
- PostgreSQL FORCE RLS remains the authoritative tenant-isolation control.
- The refactor changed dependency direction and testability, not product policy.

Current acceptance status Architecture through Phases 0-8 is implemented and characterized. The final Phase 9 independent review is intentionally separate; this document describes the architecture currently built, not a yet-to-be-reviewed future design.

3. Final Architecture

The subsystem is a compact vertical feature inside KEEL's modular monolith. Delivery code expresses HTTP concerns; application code expresses intent; orchestration coordinates the use case; deterministic functions decide scores and cold-start outcomes; narrow capabilities isolate side effects; PostgreSQL composition binds those capabilities to existing SQL helpers.



Platform authorities remain external: TenantContext, RLS, auth/entitlement, connection lifecycle

3.1 Module Responsibilities

Module	Primary responsibility	Must not own
routers/mpi.py	HTTP dependencies, entitlement guard, response/error mapping, delivery transaction lifecycle	Scoring policy, raw SQL, duplicated tenant policy
mpi/application.py	Intent-oriented operations: compute summary and get history	FastAPI delivery concerns, hidden commits, raw SQL

Module	Primary responsibility	Must not own
mpi/pipeline.py	MPI orchestration, evidence durability owner for summary	HTTP mapping, generic repository behavior
mpi/scoring.py + cold_start.py	Deterministic scoring, recency/volume/sample decisions	PostgreSQL, FastAPI, Redis, tenant context
mpi/ports.py	Narrow capability contracts	SQL implementation, commits/rollbacks
mpi/postgres.py	Default binding/composition of capabilities	Business scoring policy, transaction ownership
mpi/data_access.py + evidence.py	Existing PostgreSQL reads/validation/evidence insert	Cross-feature orchestration
mpi/errors.py	MPI-owned shared exception contract	Universal backend error framework

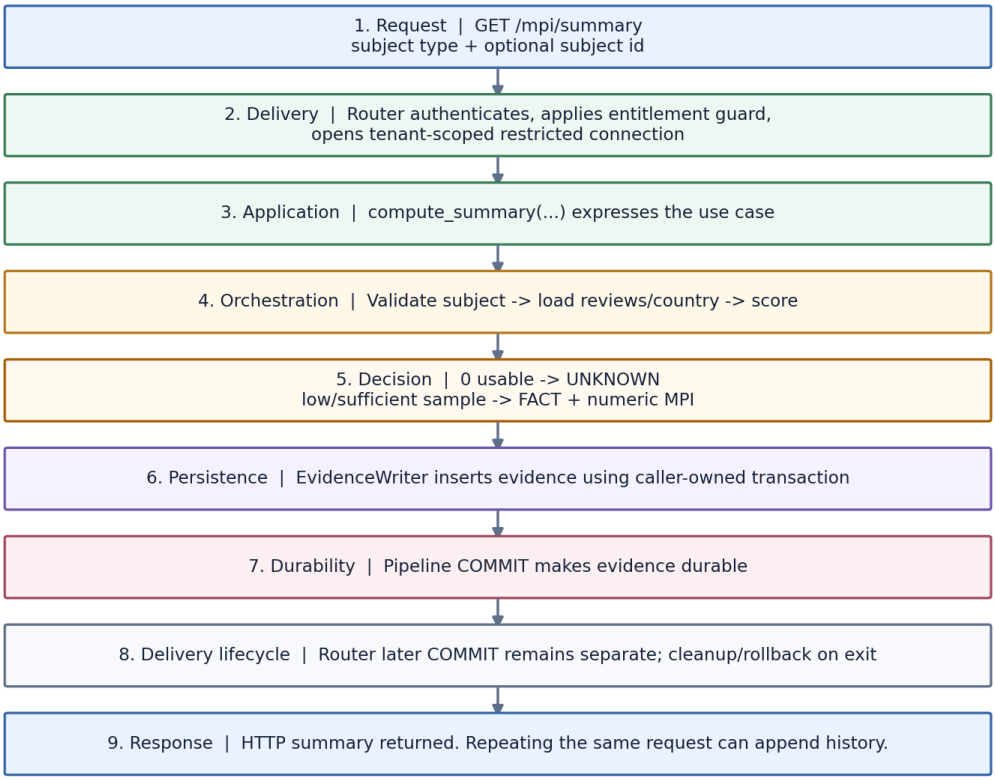
3.2 Narrow Capability Contracts

- ReviewReader - reads eligible analyzed reviews.
- SubjectValidator - validates subject existence/ownership while preserving non-disclosure semantics.
- CountryReader - reads country context when available.
- EvidenceWriter - inserts one evidence record using the caller-owned transaction.
- HistoryReader - returns ordered historical evidence rows.

Transaction rule Ports and PostgreSQL adapters do not commit, roll back, close connections or change tenant context. Transaction ownership stays explicit above them.

4. Request Flows

4.1 Summary Request



Preserved durability The pipeline commit intentionally makes evidence durable before the later router transaction finishes. If later delivery/router work fails, the already committed evidence remains durable. This is characterized current behavior, not a universal KEEL transaction pattern.

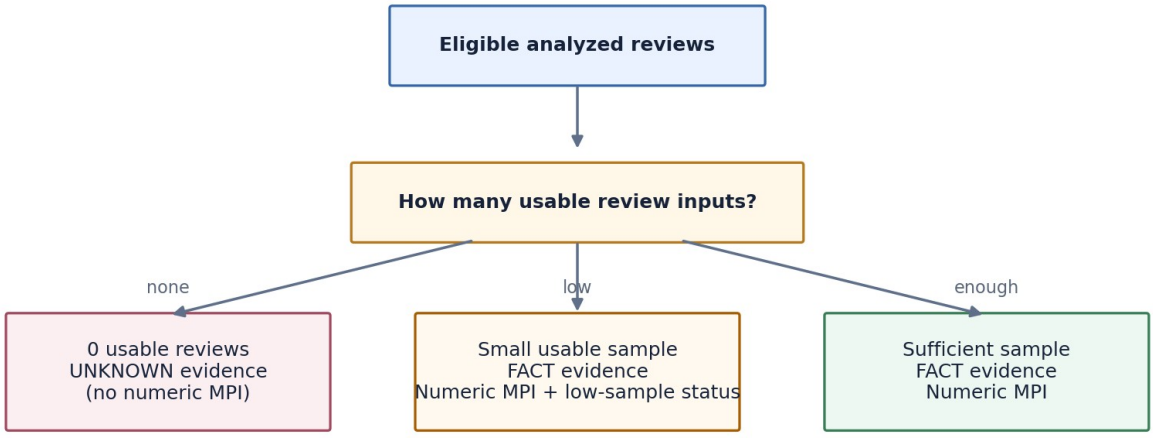
4.2 History Request

1. Router authenticates the caller, applies the existing entitlement guard and obtains a restricted tenant-scoped connection.
2. Application get_history(...) validates/canonicalizes the subject through the established capability boundary.
3. HistoryReader loads ordered persisted evidence with the existing pagination lookahead behavior.
4. Router owns the existing read transaction lifecycle and performs the current commit/cleanup behavior.
5. History is serialized back to the caller. The history path itself does not create evidence.

4.3 Current Transaction Ownership

Operation	Owner	Effect	Important consequence
Summary evidence commit	Pipeline	Makes inserted evidence durable	Later router failure cannot undo it
Later summary commit	Router	Completes later delivery transaction lifecycle	Failure can still make the HTTP request fail
History commit	Router	Completes read transaction lifecycle	No MPI evidence is written
Error rollback	Router/wrapper lifecycle	Discards pending, uncommitted work	Cannot undo evidence already committed by pipeline

5. Decision Semantics



5.1 What “usable” means at a high level

The exact eligibility/scoring rules are protected by the characterization suite. At a high level, future reviews, blocked/quarantined/blank inputs, subject mismatches and malformed sentiment data are excluded according to current policy. Rating alone does not replace missing or malformed analyzed sentiment.

5.2 Time and configuration constraints

- Python local date.today() continues to drive the default scoring/decay date behavior.
- PostgreSQL transaction time and database timezone continue to affect review eligibility/date behavior.
- MPI configuration values are still captured at module import; changing environment values after import does not change already-loaded settings without reload.
- These are preserved legacy constraints, not recommendations to copy into new features.

6. Use Cases

UC	Use case	Primary actor	System outcome	Persistence
UC-01	Compute business MPI summary	Authorized tenant member	Returns BUSINESS MPI result or UNKNOWN	Appends evidence on successful summary
UC-02	Compute product MPI summary	Authorized tenant member	Returns PRODUCT MPI result or UNKNOWN	Appends evidence on successful summary
UC-03	Compute	Authorized tenant	Returns	Appends evidence

UC	Use case	Primary actor	System outcome	Persistence
	competitor MPI summary	member	COMPETITOR MPI result or UNKNOWN	on successful summary
UC-04	Read MPI history	Authorized tenant member	Returns ordered historical FACT/UNKNOWN evidence	Read-only; no new evidence
UC-05	Handle low-data subject	Authorized tenant member	Returns UNKNOWN when no usable evidence; low sample still returns FACT + numeric MPI	Persists corresponding evidence classification
UC-06	Reject inaccessible subject	Authenticated caller without subject access	Non-disclosing failure; no cross-tenant leakage	No unauthorized evidence write
UC-07	Repeat a summary request	Authorized tenant member	Returns another summary under same current policy	May append another evidence/history row

6.1 Use-Case Contract

- Inputs: established tenant identity, subject type, optional subject ID, and existing caller-owned database connection/context.
- Application intent: compute_summary(...) or get_history(...).
- Authorities: authentication, entitlement and RLS remain external platform authorities; MPI does not duplicate them.
- Outputs: summary/history structures using the existing FACT/UNKNOWN evidence semantics.
- Failure rule: non-disclosing subject/tenant behavior and current HTTP mapping remain preserved.

7. User Scenarios

Scenario 1 - Tenant Owner checks overall business perception

Actor: Owner (same flow also applies to other characterized permitted tenant roles).

1. Owner requests the BUSINESS MPI summary for the current tenant.
2. Router validates identity/entitlement and opens a restricted tenant-scoped connection.
3. MPI validates the subject, loads eligible analyzed reviews and computes the deterministic score.
4. If usable evidence exists, MPI persists FACT evidence; if none exists, it persists UNKNOWN evidence.
5. The pipeline commits the evidence and the router finishes its separate delivery transaction lifecycle.
6. The caller receives the summary response.

Outcome Business value: one current, evidence-backed perception snapshot using tenant-isolated review data.

Scenario 2 - Manager checks a product with only a small review sample

Actor: Manager.

1. Manager requests the PRODUCT MPI summary for a product owned by the tenant.
2. The subject is validated without exposing inaccessible tenant data.
3. A small but usable analyzed-review sample is found.
4. MPI returns a numeric score and persists FACT evidence with the current low-sample status/confidence behavior.
5. The result is not converted to UNKNOWN merely because the sample is small.

Outcome Business value: users get a cautious but usable signal instead of losing all information until a large sample exists.

Scenario 3 - Staff member checks competitor perception

Actor: Staff.

1. Staff requests the COMPETITOR MPI summary for an accessible competitor subject.
2. Existing authorization/entitlement policy allows the characterized role to invoke the summary.
3. MPI uses tenant-scoped evidence only and computes the competitor result using the same deterministic core.
4. The result is persisted as evidence and returned.

Outcome Business value: the same MPI model can be applied consistently across owned and competitor subjects without weakening tenant isolation.

Scenario 4 - New subject has no usable analyzed reviews

Actor: Any currently permitted tenant member.

1. The caller requests a summary for a valid subject.
2. Eligibility filters leave zero usable analyzed review inputs.
3. MPI produces UNKNOWN rather than fabricating a score.
4. UNKNOWN evidence is persisted so the lack of sufficient evidence is itself historically visible.

Outcome Business value: KEEL represents uncertainty explicitly instead of manufacturing confidence.

Scenario 5 - User reviews MPI history

Actor: Authorized tenant member.

1. The caller requests MPI history for a valid subject.
2. Subject ownership/existence is validated through the established capability boundary.
3. HistoryReader loads persisted FACT/UNKNOWN records using the current ordering and pagination rules.
4. The request is read-only with respect to MPI evidence: no new evidence row is created.
5. The caller receives the historical records.

Outcome Business value: users can inspect how recorded perception evidence accumulated over time.

Scenario 6 - Cross-tenant or inaccessible subject is requested

Actor: Authenticated caller lacking access to the target subject.

1. The caller supplies an inaccessible or foreign-tenant subject identifier.
2. Subject validation plus PostgreSQL RLS prevents access to foreign data.
3. The failure remains non-disclosing: the response does not reveal protected subject existence/details.
4. No unauthorized evidence is created and subsequent restricted connection use remains tenant-isolated.

Outcome Security value: application checks and database RLS reinforce each other; app-level filtering does not replace RLS.

Scenario 7 - Same summary is requested repeatedly

Actor: Authorized tenant member.

1. The caller makes the same summary request multiple times against unchanged source evidence.
2. Each successful request follows the normal compute-and-persist flow.
3. Current behavior may append another evidence row even when the score is identical.
4. History therefore can grow with repeated GET /mpi/summary calls.

Outcome Important policy note: this mutating-GET/history-growth behavior is intentionally preserved for now and should not be copied automatically to other features.

8. Security & Transaction Invariants

8.1 Invariants future MPI changes must preserve unless policy explicitly changes

- PostgreSQL FORCE RLS remains authoritative for tenant isolation.
- Tenant identity comes from established platform context, not untrusted request payloads.
- Owner, Admin, Manager, Accountant and Staff retain the characterized access behavior subject to active principal and entitlement checks.
- Ports/adapters remain commit-free and connection-lifecycle-neutral.
- Summary evidence durability remains pipeline-owned; later router failure does not roll it back.
- GET /mpi/summary continues to mutate evidence/history under current policy.
- GET /mpi/history remains read-only with respect to MPI evidence.
- FACT/UNKNOWN and low-sample semantics remain unchanged unless separately approved.
- Scoring formulas, thresholds, rounding and import-time configuration behavior remain locked by tests.
- No cross-feature feature code should import router internals or MPI private infrastructure helpers.

8.2 Test safety net

The refactor built a layered safety net: Phase 0 characterization locked current behavior; later phases added deterministic, application, ports, composition and transaction-ownership tests. The latest reported focused run before final Phase 9 review passed 159 MPI tests, plus launcher tooling checks, restricted-role login and verification of 120 FORCE RLS tables on disposable test infrastructure.

9. Reference Pattern Guidance

9.1 Good patterns to reuse elsewhere in KEEL

- Start with characterization before architecture changes.
- Extract deterministic decisions before moving files.
- Put an intent-oriented application boundary between delivery and feature orchestration.
- Use narrow task-specific capabilities instead of a universal repository.
- Keep PostgreSQL composition explicit and ordinary Python; avoid a DI container for small features.
- Make transaction ownership explicit and test it with native DB behavior, not only mocks.
- Keep platform authorities (auth, tenant context, RLS, entitlements) singular instead of recreating them inside a feature.
- Move/rename files only when responsibility/dependency direction already justifies it.

9.2 MPI-specific behavior NOT to copy mechanically

- Dual summary commits and early evidence durability are feature-specific current policy.
- Mutating GET /mpi/summary and repeated history growth are preserved behavior, not universal HTTP guidance.

- Import-time configuration capture is a legacy constraint, not the preferred new-feature pattern.
- The Python-date vs PostgreSQL-time split is preserved behavior, not a recommended default.
- Five ports happen to fit MPI; other features should introduce only the capabilities they actually need.

10. Compact Implementation Map

File	Role in the final MPI slice
ai/mpi/application.py	Application operations for summary/history.
ai/mpi/pipeline.py	Summary/comparison orchestration and evidence durability boundary.
ai/mpi/scoring.py	Deterministic MPI arithmetic.
ai/mpi/cold_start.py	Deterministic sample/availability classification.
ai/mpi/ports.py	Narrow callable capability contracts.
ai/mpi/postgres.py	Default PostgreSQL capability composition.
ai/mpi/data_access.py	Existing PostgreSQL reads, validation and history query.
ai/mpi/evidence.py	Existing PostgreSQL evidence insert.
ai/mpi/errors.py	MPI-owned shared exception definitions.
ai/routers/mpi.py	FastAPI delivery, entitlement/error mapping and router-owned transaction lifecycle.

Bottom line MPI now has a clear feature boundary: HTTP delivery -> application intent -> orchestration -> deterministic decisions + narrow side-effect capabilities -> PostgreSQL composition, while preserving KEEL's tenant/security authorities and the exact transaction behavior characterized before the refactor.